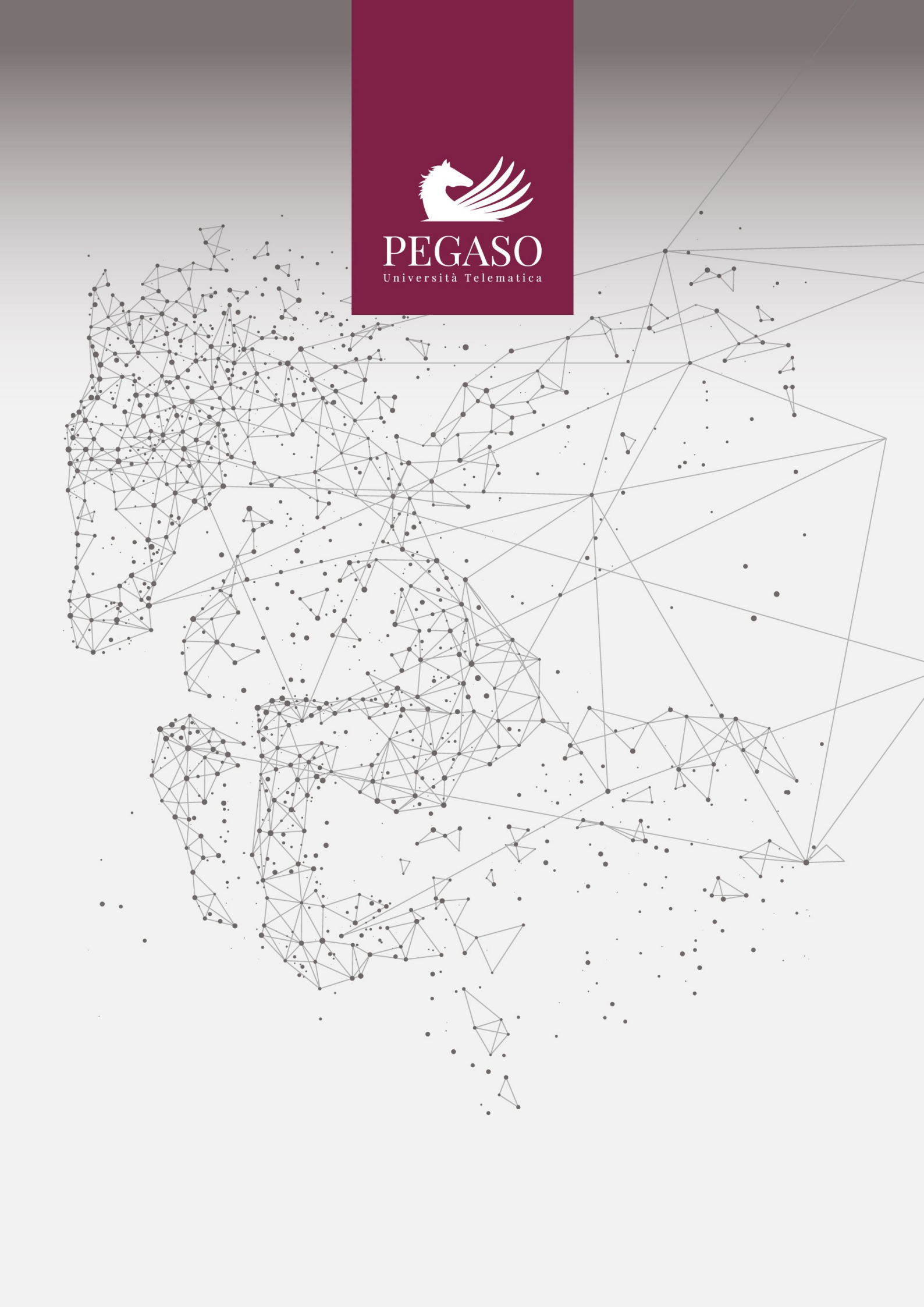

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. JAVASCRIPT 3
2. JAVASCRIPT BASICS 5
3. OBJECTS 8
4. ARRAY 12
BIBLIOGRAFIA E SITOGRAFIA 16

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Javascript

Abbiamo 3 possibilità di inserire tale script nella pagina:

1. Nell'head (posizione 1)
2. Nel body subito dopo il tag body (posizione 2)
3. In un punto qualunque del body (posizione 3)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>TEST</title>
  <!-- POSIZIONE 1 -->
</head>
<body>
  <!-- POSIZIONE 2 -->
  <p id="myP">Hello world</p>
  <!-- POSIZIONE 3 -->
</body>
</html>
```

1. Se lo script si trova nell'head, questo viene invocato immediatamente; se questo dipende da elementi specifici della pagina HTML lo script potrebbe non funzionare correttamente
2. Se lo script si trova nel body, questo viene invocato immediatamente durante il caricamento della pagina rispettando il suo posizionamento; se lo script dipende da elementi specifici della pagina HTML che si trovano DOPO (a livello di posizionamento) il caricamento dello script, lo script potrebbe non funzionare correttamente
3. Se lo script si trova nel body subito prima la chiusura del tag stesso, siamo certi che tutti gli elementi della pagina sono stati caricati, pertanto se lo script dipende da elementi specifici della pagina HTM, siamo certi che lo script funzionerà correttamente

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È inoltre possibile scrivere il codice Javascript all'interno di un file esterno (tipicamente con estensione .js) e richiamarlo in una delle posizioni indicate in precedenza, attraverso la sintassi:

```
<script src="script.js"></script>
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Javascript basics

La sintassi JavaScript definisce due tipi di valori:

- Fissi (fixed)
- Variabili (variables)

I valori fissi sono chiamati letterali (literals) mentre quelli variabili sono chiamati variabili (variabiles). Le due regole di sintassi più importanti per i valori fissi sono:

- I numeri sono scritti con o senza decimali
- Le stringhe sono scritte tra singoli i doppi apici

In JavaScript, sia la parola chiave **var** che la parola chiave **let** vengono utilizzate per dichiarare variabili, ma hanno alcune differenze importanti. La principale differenza tra var e let è che le variabili dichiarate con var hanno un ambito di funzione (function scope), mentre le variabili dichiarate con let hanno un ambito di blocco (block scope). Altre peculiarità del let sono:

- le variabili definite con let non possono essere ridefinite
- le variabili definite con let devono essere dichiarate prima dell'utilizzo

JavaScript ha 8 tipi di dato:

1. String
2. Number: double (floating point a 64 bit)
3. Bigint: numeri interi più grandi di "Number"
4. Boolean
5. Undefined: una variabile senza valore
6. Null
7. Symbol
8. Object

Le istruzioni condizionali vengono utilizzate per eseguire azioni diverse in base a diverse condizioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In JavaScript abbiamo le seguenti istruzioni condizionali:

- Usare **if** per specificare un blocco di codice da eseguire, se una condizione specificata è vera
- Usare **else** per specificare un blocco di codice da eseguire, se la stessa condizione è falsa
- Usare **else if** per specificare una nuova condizione da testare, se la prima condizione è falsa
- Usare **switch** per specificare molti blocchi alternativi di codice da eseguire

JavaScript supporta diversi tipi di cicli:

- **for** - cicla attraverso un blocco di codice un numero di volte
- **for/in** - cicla attraverso le proprietà di un oggetto
- **for/of** - cicla attraverso i valori di un oggetto iterabile
- **while** - cicla attraverso un blocco di codice finché una specifica condizione è vera
- **do/while** - cicla anche attraverso un blocco di codice finché una specifica condizione è vera

Lo statement **for** crea un ciclo con 3 espressioni opzionali:

```
for (espressione1; espressione2; espressione3) {
// blocco di codice da eseguire
}
```

- L'espressione1 viene eseguita (una sola volta) prima dell'esecuzione del blocco di codice.
- L'espressione2 definisce la condizione per l'esecuzione del blocco di codice.
- L'espressione3 viene eseguita (ogni volta) dopo l'esecuzione del blocco di codice.

Il ciclo **while** esegue un blocco di codice fintanto che una determinata condizione risulta essere vera. La sintassi del ciclo while è la seguente:

```
while (condizione) {
  // blocco di codice da eseguire
}
```

La condizione è un'espressione che viene valutata all'inizio di ogni iterazione del ciclo. Se la condizione è vera, il blocco di codice all'interno del ciclo viene eseguito. Dopo l'esecuzione del blocco di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

codice, la condizione viene valutata nuovamente. Questo processo continua fino a quando la condizione diventa falsa, momento in cui il ciclo while termina e l'esecuzione del programma continua al di fuori del ciclo.

Il ciclo **do...while** è simile al ciclo while, ma ha una differenza significativa: il blocco di codice all'interno del ciclo viene eseguito almeno una volta, indipendentemente dalla verità della condizione. La sintassi del ciclo do...while è la seguente:

```
do {
  // blocco di codice da eseguire
} while (condizione);
```

Il blocco di codice all'interno del ciclo viene eseguito una volta senza verificare la condizione. Dopo l'esecuzione del blocco di codice, la condizione viene valutata. Se la condizione è vera, il blocco di codice viene eseguito di nuovo e il processo continua finché la condizione diventa falsa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Objects

In JavaScript, un oggetto è una struttura di dati complessa e flessibile utilizzata per rappresentare e organizzare i dati. È un contenitore per un insieme di proprietà (che possono essere variabili, funzioni, altri oggetti, ecc.) che possono essere accedute e manipolate tramite il nome della proprietà o l'indice.

Gli oggetti sono spesso utilizzati per rappresentare entità del mondo reale come persone, automobili, case, ecc., ma possono essere utilizzati anche per organizzare il codice e definire moduli e librerie riutilizzabili. Gli oggetti in JavaScript sono dinamici, il che significa che le proprietà possono essere aggiunte o rimosse in qualsiasi momento durante l'esecuzione del programma.

Ecco alcuni esempi di oggetti in JavaScript:

Un oggetto persona con proprietà come nome, età e indirizzo:

```
const person = {
  name: 'John Doe',
  age: 30,
  address: {
    street: '123 Main St',
    city: 'Anytown',
    state: 'CA'
  }
};
```

Un oggetto auto con proprietà come marca, modello, anno di fabbricazione e metodi per accendere/spegnere l'auto:

```
const car = {
  make: 'Honda',
  model: 'Civic',
  year: 2020,
  isOn: false,
  turnOn: function() {
    this.isOn = true;
    console.log('The car is on.');
  },
  turnOff: function() {
    this.isOn = false;
    console.log('The car is off.');
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  }
};
```

Un oggetto utente con proprietà come nome utente, email e un array di post recenti:

```
const user = {
  username: 'johndoe',
  email: 'johndoe@gmail.com',
  posts: [
    { title:'First post',content:'Lorem ipsum.' },
    { title:'Second post',content:'Nulla quis lorem' },
    { title:'Third post',content: 'Sed porttitor lectus nibh.' }
  ]
};
```

In ogni caso, gli oggetti in JavaScript possono avere proprietà di diversi tipi come stringhe, numeri, booleani, oggetti annidati, funzioni e altro ancora.

Per accedere alle proprietà di un oggetto in JavaScript, si può utilizzare la notazione punto (**dot notation**) o la notazione delle parentesi quadre (**bracket notation**).

La notazione punto viene utilizzata per accedere alle proprietà che hanno un nome valido in JavaScript (come una stringa senza spazi), mentre la notazione delle parentesi quadre viene utilizzata per accedere alle proprietà che hanno un nome non valido o dinamico (come una stringa con spazi o il cui nome viene calcolato dinamicamente):

```
console.log(person.name);
console.log(person.address.city);
console.log(person['full name']);
console.log(person['age']);
```

Il loop **for-in** in JavaScript viene utilizzato per iterare le proprietà di un oggetto. In ogni iterazione, il loop assegna il nome della proprietà corrente alla variabile di iterazione.

La sintassi del loop **for-in** è la seguente:

```
for (var propertyName in object) {
  // code to be executed
}
```

Dove propertyName è il nome della proprietà corrente e object è l'oggetto da iterare:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
const person = {
  name: 'stefano',
  age: 45,
  address: {
    street: 'la mia strada 123',
    city: 'City',
    state: 'IT'
  }
};

for (var propertyName in person) {
  console.log(propertyName + ': ' + person[propertyName]);
}
```

In JavaScript, this è una keyword che fa riferimento all'oggetto corrente, ovvero l'oggetto su cui è stata chiamata la funzione o il metodo. Il valore di this dipende dal contesto in cui viene utilizzato e può assumere diversi valori.

Nella maggior parte dei casi, il valore di this viene determinato durante l'esecuzione del codice e dipende dal modo in cui viene chiamata la funzione o il metodo. Ad esempio, se una funzione viene chiamata come metodo di un oggetto, il valore di this sarà l'oggetto stesso. Ecco un esempio:

```
const persona = {
  nome: "Mario",
  saluta: function() {
    console.log("Ciao, mi chiamo " + this.nome);
  }
};

persona.saluta(); // Output: "Ciao, mi chiamo Mario"
```

In questo esempio, la funzione saluta è un metodo dell'oggetto persona e viene chiamata utilizzando la notazione puntata. Quando la funzione viene eseguita, il valore di this diventa l'oggetto persona.

Tuttavia, ci sono alcune eccezioni in cui il valore di this non viene determinato dinamicamente ma viene fissato dal codice. Ad esempio, se una funzione viene chiamata con il metodo call o apply, è possibile specificare il valore di this come primo parametro. Ecco un esempio:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
function saluta() {
  console.log("Ciao, mi chiamo " + this.nome);
}

const persona1 = {
  nome: "Mario"
};

const persona2 = {
  nome: "Luigi"
};

saluta.call(persona1); // Output: "Ciao, mi chiamo Mario"
saluta.call(persona2); // Output: "Ciao, mi chiamo Luigi"
```

In questo caso, la funzione saluta viene chiamata utilizzando il metodo call e il primo parametro specifica il valore di this durante l'esecuzione della funzione.

In generale, il valore di this è un concetto importante in JavaScript e può essere utilizzato per accedere alle proprietà e ai metodi degli oggetti correnti durante l'esecuzione del codice.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Array

In JavaScript, un array è un oggetto che permette di memorizzare una sequenza di valori in una singola variabile. Un array può contenere valori di qualsiasi tipo di dato, come numeri, stringhe, oggetti o altri array.

Per creare un array in JavaScript, si può utilizzare la seguente sintassi:

```
var nomeArray = [valore1, valore2, ..., valoreN];
```

Dove nomeArray è il nome dell'array e valore1, valore2, valoreN sono i valori che si vogliono memorizzare nell'array.

Ad esempio, si può creare un array di numeri come segue:

```
var numeri = [1, 2, 3, 4, 5];
```

Oppure un array di stringhe:

```
var frutti = ['mela', 'banana', 'arancia'];
```

Gli array in JavaScript sono oggetti ordinati, il che significa che i valori sono memorizzati in ordine numerico, a partire dall'indice 0. Si può accedere ai singoli valori dell'array utilizzando la notazione delle parentesi quadre e l'indice corrispondente:

```
var numeri = [1, 2, 3, 4, 5];
console.log(numeri[0]); // output: 1
console.log(numeri[3]); // output: 4
```

Si può anche modificare i valori di un array assegnando un nuovo valore a un indice specifico:

```
var frutti = ['mela', 'banana', 'arancia'];
frutti[1] = 'uva';
console.log(frutti); // output: ['mela', 'uva', 'arancia']
```

Ecco due esempi di come utilizzare il ciclo for e il metodo forEach per iterare su un array:

```
var numeri = [1, 2, 3, 4, 5];
// Utilizzo del ciclo for per iterare sull'array
for (var i = 0; i < numeri.length; i++) {
  console.log(numeri[i]);
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, viene utilizzato il ciclo for per iterare sull'array numeri. La variabile i viene inizializzata a 0 e viene incrementata ad ogni iterazione del ciclo. Il ciclo viene eseguito finché i è minore della lunghezza dell'array numeri.

Durante ogni iterazione, viene visualizzato il valore dell'elemento corrente dell'array utilizzando la notazione delle parentesi quadre.

```
var frutti = ['mela', 'banana', 'arancia'];
// Utilizzo del metodo forEach per iterare sull'array
frutti.forEach(function(frutto) {
  console.log(frutto);
});
```

In questo esempio, viene utilizzato il metodo forEach per iterare sull'array frutti. Il metodo forEach accetta una funzione di callback che viene eseguita per ogni elemento dell'array. La funzione di callback riceve come parametro l'elemento corrente dell'array. Durante ogni iterazione, viene visualizzato il valore dell'elemento corrente dell'array utilizzando la variabile frutto che rappresenta l'elemento corrente.

In JavaScript, i metodi **push** e **pop** sono utilizzati per aggiungere o rimuovere elementi da un array. Il metodo push viene utilizzato per aggiungere uno o più elementi alla fine di un array, mentre il metodo pop viene utilizzato per rimuovere l'ultimo elemento di un array.

Ecco degli esempi di come utilizzare questi metodi:

```
var numeri = [1, 2, 3];
// Utilizzo del metodo push per aggiungere un elemento alla fine dell'array
numeri.push(4);
console.log(numeri); // output: [1, 2, 3, 4]
// Utilizzo del metodo push per aggiungere più elementi alla fine dell'array
numeri.push(5, 6);
console.log(numeri); // output: [1, 2, 3, 4, 5, 6]
```

In questo esempio, viene utilizzato il metodo push per aggiungere uno o più elementi alla fine dell'array numeri. Il metodo push accetta uno o più argomenti che rappresentano gli elementi da

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

aggiungere all'array. Nel primo caso viene aggiunto il valore 4 alla fine dell'array, mentre nel secondo caso vengono aggiunti i valori 5 e 6.

```
var frutti = ['mela', 'banana', 'arancia'];
// Utilizzo del metodo pop per rimuovere l'ultimo elemento dell'array
var ultimoFrutto = frutti.pop();
console.log(frutti); // output: ['mela', 'banana']
console.log(ultimoFrutto); // output: 'arancia'
```

In questo esempio, viene utilizzato il metodo pop per rimuovere l'ultimo elemento dell'array frutti. Il metodo pop non accetta alcun argomento e rimuove semplicemente l'ultimo elemento dell'array. Nel caso di questo esempio, il valore 'arancia' viene rimosso dall'array frutti e assegnato alla variabile ultimoFrutto. Infine, vengono visualizzati sia l'array frutti modificato che il valore dell'ultimo elemento rimosso.

La funzione **splice** viene utilizzata per aggiungere o rimuovere elementi da un array in una posizione specifica. Il suo utilizzo è il seguente:

```
array.splice(start, deleteCount, item1, item2, ...)
```

- start: l'indice a cui iniziare ad aggiungere o rimuovere elementi
- deleteCount: il numero di elementi da rimuovere a partire dall'indice start
- item1, item2, ecc.: gli elementi da aggiungere all'array, partendo dall'indice start

Ecco alcuni esempi:

```
var numeri = [1, 2, 3, 4, 5];
// Rimuove un elemento a partire dall'indice 2
numeri.splice(2, 1);
console.log(numeri); // output: [1, 2, 4, 5]
// Aggiunge un elemento all'indice 2 e rimuove un elemento a partire
dall'indice 3
numeri.splice(2, 1, 6);
console.log(numeri); // output: [1, 2, 6, 5]
// Aggiunge due elementi all'indice 2 e non rimuove alcun elemento
numeri.splice(2, 0, 7, 8);
console.log(numeri); // output: [1, 2, 7, 8, 6, 5]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, la funzione splice viene utilizzata per rimuovere un elemento dall'array, aggiungere un elemento all'array e aggiungere due elementi all'array.

Utilizzo della funzione slice

La funzione **slice** viene utilizzata per creare una nuova copia di un array, partendo da un indice e terminando in un altro indice. Il suo utilizzo è il seguente:

```
array.slice(start, end)
```

- start: l'indice a cui iniziare a copiare l'array
- end: l'indice a cui terminare la copia dell'array (non incluso)

Ecco un esempio:

```
var numeri = [1, 2, 3, 4, 5];
// Crea una copia di una porzione dell'array originale
var numeriCopia = numeri.slice(2, 4);
console.log(numeriCopia); // output: [3, 4]
```

In questo esempio, la funzione slice viene utilizzata per creare una copia dell'array numeri, partendo dall'indice 2 e terminando all'indice 4 (non incluso).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005
- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).